

Build a Resume and GitHub Portfolio

A useful portfolio answers three questions: what problem did you solve, what did you personally build, and how can someone verify it? Align the resume, repository, demo, and interview narrative so a reviewer sees one consistent, defensible story.

For: Engineers who want their public work and resume to communicate real ability quickly, without unsupported impact claims, copied projects, or exposure of private employer information.

Suggested pace: 2-3 weeks, followed by regular updates | All experience levels

What you will be able to do

- Write concise experience bullets with truthful scope, action, and evidence.
- Prepare a repository that a new reader can run and inspect safely.
- Choose projects that demonstrate different relevant engineering skills.
- Audit public artifacts for secrets, broken links, and misleading claims.

Build the right mental model

Lead with relevance

Order projects and experience by fit to the target role, not by the number of tools used. A backend role benefits from contracts, persistence, reliability, and tests; a frontend role also needs interaction quality, accessibility, and performance evidence.

Make claims falsifiable

Explain the workload and measurement behind any speed or reliability improvement. If there was no measurement, describe the functional change honestly. Distinguish a local benchmark from production impact and your contribution from the team's overall outcome.

A README is part of the product

Include purpose, prerequisites, setup, sample input, expected output, tests, architecture, and known limitations. Do not require private credentials to evaluate the basic demo. Provide synthetic fixtures and document optional external integrations separately.

Study with evidence.

Select two strong projects rather than polishing ten incomplete repositories. Review each from the perspective of a stranger with no context. Automate checks for setup and tests where possible, then ask a peer to attempt the instructions without help.

Days 1-3: Audit and select

Inventory public repositories, links, deployment status, and resume claims. Remove accidental secrets through the appropriate revocation process; deleting a file alone does not invalidate a leaked credential. Choose projects with clear role relevance.

Deliverable and check: A short artifact inventory with owners, status, and a prioritized cleanup list.

Days 4-7: Document reproducibility

Write setup instructions and sample data. Pin important dependency versions, explain environment variables without values, and add a small automated test command. Include the simplest successful workflow and one failure case.

Deliverable and check: A peer can clone and run a demo using only public instructions and synthetic inputs.

Week 2: Strengthen evidence

Add meaningful tests, one design decision record, and a measured improvement if justified. Update resume bullets to match the actual repository and your contribution. Keep text skimmable and avoid keyword stuffing.

Deliverable and check: A defensible problem-action-result story with reproducible evidence or clearly stated qualitative outcomes.

Week 3: Review and maintain

Check mobile presentation, accessibility, downloads, and all links. Rehearse a project walkthrough and answer why you chose the design. Schedule periodic dependency and deployment checks so the portfolio does not silently decay.

Deliverable and check: A final reviewer checklist and a consistent narrative across resume, GitHub, and interview answers.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Turn a vague project into a verifiable case study

A repository says 'scalable URL shortener using modern technologies' but contains no workload, tests, or discussion of collisions. Replace the claim with a small, evidence-backed engineering story.

A worked approach

1. Define create-link, redirect, and expiration behavior. State the storage model and what a collision means. Show a sample request and response, plus a clear list of features that are not implemented.
2. Add tests for duplicate aliases, malformed destinations, missing links, and expired links. If you run a load test, record hardware, concurrency, dataset, cache state, and latency distribution instead of publishing one unexplained requests-per-second number.
3. Write a decision note comparing random IDs with sequential encoding and explaining abuse controls and redirect semantics. Update the resume to describe the implemented scope, not a hypothetical global service.

Expected result

The project becomes credible because its behavior, evidence, and limitations are visible. A smaller truthful system is easier to defend than a large claim supported only by a technology diagram.

Verify the result

- Do README commands work in a clean environment?
- Can every numerical claim be traced to a method and result?
- Are secrets, private URLs, and customer data absent from both files and shared examples?

Extend the exercise

Ask a reviewer to select one edge case you did not anticipate. Add the smallest test and implementation change, then document why the original assumption was incomplete.

Make the learning stick.

Should I include every technology I have tried?

No. List skills you can discuss and demonstrate at the relevant level. A short credible list helps an interviewer choose meaningful questions; a large unsupported list creates avoidable credibility risk.

What if my best work is confidential?

Describe the problem class, your role, and permitted outcomes at an appropriate level of abstraction. Build a separate synthetic example if useful. Never publish proprietary code, private architecture, or customer information to prove experience.

Are polished screenshots enough?

They help communicate the result, but do not replace setup instructions, tests, and reasoning. A reviewer should understand both the user-facing behavior and the engineering work behind it.

Avoid these traps

- Publishing secrets or assuming deletion removes them from history.
- Calling a local demo highly scalable without a workload or evidence.
- Using generic AI-generated claims that do not match the repository.

Ready for the next step?

- All links and setup instructions have been checked.
- Each project has a clear scope and at least one meaningful test.
- Resume metrics and ownership claims are supportable.
- Public artifacts contain only material you are authorized to share.

Reliable next reads

- [GitHub: README guidance](#)
- [GitHub: removing sensitive data](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.